

Aufgabe 1 (16 Punkte)

Diese Aufgabe besteht aus drei Teilaufgaben.

1(a):

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie ein generisches Functional **public**-Interface **A** mit dem generischen Typparameter **C**. **C** ist beschränkt auf **Container** und alle Subtypen von **Container**.

Default-Methode **fnc**: ist generisch mit einem Typparameter **T**, der auf **Number** und die Subtypen von **Number** beschränkt ist. **fnc** verfügt über eine Rückgabe vom Typ **boolean**. **fnc** verfügt über zwei Parameter. (1) **ason** vom generischen Typ **HashSet**, welcher auf **Number** und alle Supertypen von **Number** beschränkt ist. (2) **t** vom formalen Typ **T**. Falls **t** nicht in **ason** enthalten ist, soll **t** dem **HashSet** hinzugefügt werden. Andernfalls soll **t** aus dem **HashSet** entfernt werden. Die Methode gibt die Information zurück, ob **t** in **ason** enthalten war.

Funktionale Methode **apply**: hat einen Parameter **f** vom formalen Typ **DoubleFunction** mit dem generischen Typen **Integer** und einen Parameter **d** vom Typ **Double**. **apply** hat einen Rückgabewert vom Typ **Integer**.

Hinweis: Sie müssen hier Restricted Genericity von Wildcards unterscheiden.

Erinnerung: Die Methode **contains** von **HashSet** prüft, ob der gegebene Parameter im spezifischen **HashSet** enthalten ist. Diese Information wird als **boolean** zurückgegeben. Hierfür benötigt **contains** den zu überprüfenden Parameter.

Die Methode **add** von **HashSet** fügt ihren Parameter dem **HashSet** hinzu. **add** benötigt hierfür den einzufügenden Parameter. **add** kann als **void**-Methode verwendet werden.

Die Methode **remove** von **HashSet** entfernt den übergebenen Parameter aus dem **HashSet**. **set** kann als **void**-Methode verwendet werden.

Referenzlösung:

```
public interface A<C extends Container> {
    default <T extends Number> boolean fnc(HashSet<? super Number>
ason, T t) {
        // Alternativ: Prüfung in if und return getrennt möglich
        boolean contains = ason.contains(t);
        if (!contains) {
            ason.add(t);
        } else {
            ason.remove(t);
        }
        return contains;
    }
}
```

```
Integer apply(DoubleFunction<Integer> f, Double d);  
}
```

Bewertungsschema:

A: Der Kopf von A ist insgesamt erkennbar korrekt.

Anmerkung: Sollten hier "nur" die generischen Typparameter inkorrekt, diese aber in (b) oder (c) korrekt sein, wird dieser Punkt hier auch vergeben.

B: Methode apply ist erkennbar korrekt.

Anmerkung: Sollte apply hier nicht korrekt, aber der Kopf von apply in (b) korrekt sein, wird dieser Punkt hier auch vergeben.

C: Der generische Typparameter von fnc ist erkennbar korrekt.

D: Der Parameter ason von fnc ist erkennbar korrekt.

E: Der Parameter t von fnc ist erkennbar korrekt.

F: Der Rumpf von fnc ist erkennbar korrekt.

G: Nur zu vergeben, wenn auch alle anderen Punkte vergeben werden und auch darüber hinaus nichts falsch ist: Die gesamte Lösung ist erkennbar syntaktisch, semantisch und logisch vollständig korrekt.

1(b):

Erinnerung 1: Der Typparameter von **A** aus Teilaufgabe (a) heißt **C** und ist auf **Container** und alle Subtypen von **Container** beschränkt.

Erinnerung 2: Methode **apply** von **A** hat einen Parameter **f** vom formalen Typ **DoubleFunction** mit dem Typen **Integer** und einen Parameter **d** vom Typ **Double**. **apply** hat einen Rückgabewert vom Typ **Integer**. Die Anwendungsfunktion von **DoubleFunction** heißt **apply**.

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie eine **public**-Klasse **B**, die das Interface **A** und das Interface **ContainerListener** (**java.awt.event.ContainerListener**) implementiert.

Methode **apply** wendet den ersten Parameter auf den zweiten an und liefert das Resultat zurück. Methoden des Interface **ContainerListener** sind in **B** nicht implementiert.

Referenzlösung:

```
abstract public class B<C extends Container> implements A<C>,
ContainerListener {
    public Integer apply(DoubleFunction<Integer> f, Double d) {
        return f.apply(d);
    }
}
```

Bewertungsschema:

A: Die implements-Klausel mit den beiden implementierten Interfaces ist erkennbar korrekt gebildet.

B: A nicht nochmals werten: Der Kopf der Klasse ist erkennbar korrekt (insbesondere abstract).

C: Der Kopf der Methode apply ist erkennbar korrekt (insbesondere public).

D: Der Rumpf der Methode apply ist erkennbar korrekt.

E: Nur zu vergeben, wenn auch alle anderen Punkte vergeben werden und auch darüber hinaus nichts falsch ist: Die gesamte Lösung ist erkennbar syntaktisch, semantisch und logisch vollständig korrekt.

1(c):

Erinnerung 1: Der Typparameter von **A** aus Teilaufgabe (a) heißt **C** und ist auf **Container** und alle Subtypen von **Container** beschränkt.

Erinnerung 2: **B** implementiert **ContainerListener**. Die Methode **componentAdded** von **ContainerListener** hat einen Parameter **e** vom formalen Typ **ContainerEvent** und verfügt über keine Rückgabe. Die parameterlose Methode **getID** von **ContainerEvent** hat den Rückgabotyp **int**.

Hinweis 1: Das Interface **ContainerListener** verfügt neben der Methode **componentAdded** zusätzlich über die Methode **componentRemoved**. **componentRemoved** soll in der implementierten Klasse nicht integriert werden.

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie eine **public-Klasse D**, die direkt von **B** abgeleitet ist und ein **private**-Objektattribut **id** vom primitiven Datentyp **int** hat. Implementieren Sie die Methode **componentAdded** von **ContainerListener** so, dass sie die Rückgabe der Methode **getID** des **ContainerEvent** nach **id** kopiert.

Referenzlösung:

```
abstract public class D<C extends Container> extends B<C> {
    private int id;
    public void componentAdded(ContainerEvent e) {
        id = e.getID();
    }
}
```

Bewertungsschema:

A: Die Methode **componentAdded** ist erkennbar korrekt (insbesondere **public**).

B: Der Kopf der Klasse ist erkennbar korrekt (insbesondere **abstract**).

C: Das Attribut **id** ist erkennbar korrekt definiert.

D: Nur zu vergeben, wenn auch alle anderen Punkte vergeben werden und auch darüber hinaus nichts falsch ist: Die gesamte Lösung ist erkennbar syntaktisch, semantisch und logisch vollständig korrekt.

Aufgabe 2 (22 Punkte)

Diese Aufgabe besteht aus drei Teilaufgaben.

2(a):

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Gegeben seien eine direkt von Klasse **Exception** abgeleitete Klasse **Exc1** und eine von **Exc1** direkt abgeleitete Klasse **Exc2**. Dabei habe **Exc1** einen parameterlosen **public**-Konstruktor und **Exc2** habe einen **public**-Konstruktor mit einem Parameter vom formalen Typ **int**.

Schreiben Sie eine **public**-Klasse **Exc3**, die direkt von **Exc2** abgeleitet ist. Klasse **Exc3** soll einen **public**-Konstruktor mit einem Parameter **lst** vom formalen Typ **List<Integer>** haben. Falls **lst** gleich **null** ist oder -- im Fall ungleich **null** -- die Liste **lst** leer ist, soll der Konstruktor von **Exc3** den Konstruktor von **Exc2** mit 0 aufrufen, andernfalls mit dem ersten Element in **lst**.

Hinweis: Wie Sie wissen, darf **super** nur einmal aufgerufen werden. Sie müssen also die Fallunterscheidung in den aktuellen Parameter von **super** integrieren.

Erinnerung: Zur Überprüfung, ob eine Liste leer ist, hat **List<E>** die parameterlose boolesche Methode **isEmpty**.

Die Methode **get** von **List<E>** liefert das Element an dem Index zurück, der durch den aktuellen Parameter angegeben ist (wie üblich beginnend mit 0).

Referenzlösung:

```
public class Exc3 extends Exc2 {
    public Exc3 ( List<Integer> lst ) {
        super ( lst == null || lst.isEmpty() ? 0 : lst.get(0) );
        // Alternativ: lst == null ? 0 : lst.isEmpty() ? 0 : lst.get(0)
        // Alternativ statt lst.get(0): lst.get(0).intValue()
    }
}
```

Bewertungsschema:

A: Die Fallunterscheidung mittels Bedingungsoperator ist erkennbar syntaktisch und semantisch korrekt.

B: Die Fallunterscheidung mittels Bedingungsoperator ist erkennbar logisch korrekt.

C: A+B nicht nochmals werten: Der Aufruf von **super** ist erkennbar korrekt.

D: Die extends-Klausel ist erkennbar korrekt.

E: Nur zu vergeben, wenn auch alle anderen Punkte vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

2(b):

Erinnerung: **Exc1** ist direkt von **Exception**, **Exc2** ist direkt von **Exc1** und **Exc3** ist direkt von **Exc2** abgeleitet. Klasse **Exc1** hat einen parameterlosen **public**-Konstruktor, **Exc2** einen mit einem Parameter vom formalen Typ **int** und **Exc3** einen mit einem Parameter vom formalen Typ **List<Integer>**. Klasse **Optional<T>** hat eine parameterlose Methode **isEmpty** sowie eine parameterlose Methode **get**, die bei einem nicht-leeren **Optional**-Objekt einen Verweis auf das enthaltene **T**-Objekt zurückliefert. Die parameterlose Methode **intValue** von **Integer** liefert die eingekapselte Zahl zurück.

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie eine **public**-Klassenmethode **m** mit Rückgabotyp **int**, einem Parameter **lst** vom formalen Typ **List<Integer>** und einem Parameter **i** vom formalen Typ **Optional<Integer>**. Die Klasse, zu der **m** gehört, müssen Sie nicht schreiben.

Falls **lst** nicht auf ein Objekt verweist, soll **m** eine **Exc1** werfen. Andernfalls: Falls **i** nicht auf ein Objekt verweist oder leer ist, wirft **m** eine **Exc2**, wobei der Konstruktor von **Exc2** mit 1 aufgerufen wird. Im verbliebenen Fall soll Folgendes passieren: Falls die in **i** enthaltene Zahl kleiner als 0 ist, soll **m** eine **Exc3** mit **lst** werfen; andernfalls liefert **m** die in **i** enthaltene Zahl zurück.

Verbindliche Anforderung: Die **throws**-Klausel soll genau eine Exception-Klasse enthalten. Diese darf aber nicht Klasse **Exception** oder Klasse **Throwable** sein (sonst Punktabzug).

Referenzlösung:

```
public static int m ( List<Integer> lst, Optional<Integer> i ) throws Exc1
{
    if ( lst == null )
        throw new Exc1 ();
    if ( i == null || i.isEmpty() )    // Alternativ: zwei separate if's
        throw new Exc2 ( 1 );
    int v = i.get().intValue();    // Auto-Unboxing auch ok
    if ( v < 0 )                    // i.get() < 0 auch ok
        throw new Exc3 ( lst );
    return v;
}
```

Anmerkungen:

1. Die Referenzlösung folgt 1:1 der logischen Struktur der Aufgabenstellung. Logisch äquivalente Lösungen mit anderer Struktur sind selbstverständlich ebenfalls ok (sofern korrekt gebildet). Dies schließt natürlich auch die geeignete Ersetzung von if-Verzweigungen durch Ausdrücke mit Bedingungsoperator ein.
2. Anstelle der Zwischenspeicherung der in **i** enthaltenen Zahl darf auch 2x **get** aufgerufen werden.

Bewertungsschema:

A: Die throws-Klausel ist erkennbar korrekt und darf insbesondere nur genau eine Exception-Klasse enthalten. Diese dürfen aber nicht Exception oder Throwable sein.

B: A nicht nochmals werten: Der Kopf der Methode m ist erkennbar korrekt (insbesondere static).

C: Der Fall `lst == null` wird erkennbar korrekt getestet und behandelt.

D: Der Fall `i == null` wird erkennbar korrekt getestet und behandelt.

E: Der Fall, dass i leer ist, wird erkennbar korrekt getestet und behandelt.

F: Der Fall, dass i eine Zahl kleiner als 0 enthält, wird erkennbar korrekt getestet und behandelt.

G: Der Fall, dass i eine Zahl größer oder gleich 0 enthält, wird erkennbar korrekt getestet und behandelt.

H: Nur zu vergeben, wenn auch alle anderen Punkte vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

2(c):

Erinnerung: **Exc1** ist direkt von **Exception**, **Exc2** ist direkt von **Exc1** und **Exc3** ist direkt von **Exc2** abgeleitet. Klasse **Exc1** hat einen parameterlosen **public**-Konstruktor, **Exc2** einen mit einem Parameter vom formalen Typ **int** und **Exc3** einen mit einem Parameter vom formalen Typ **List<Integer>**. Die Klassenmethode **m** aus Teilaufgabe (b) hat Rückgabotyp **int**, einen Parameter vom formalen Typ **List<Integer>** und einen zweiten Parameter vom formalen Typ **Optional<Integer>**. Die Klassenmethode **of** von **Optional<T>** erzeugt ein **Optional<T>**-Objekt mit ihrem Parameter vom formalen Typ **T**.

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie eine **public**-Objektmethode **m2** mit Rückgabotyp **int** und einem Parameter **lst** vom formalen Typ **List<Integer>**. Methode **m2** gehört nicht zur selben Klasse wie Methode **m**. Die Klasse, zu der **m** gehört, heie **X**. Die Klasse, zu der **m2** gehört, mssen Sie nicht schreiben.

In **m2** wird die Methode **m** aus Teilaufgabe (b) aufgerufen. Der erste aktuelle Parameter bei diesem Aufruf ist **lst**; der zweite aktuelle Parameter ist nicht leer und enthlt den Wert 1. Falls **m** bei diesem Aufruf eine **Exc1** wirft, soll **m2** eine **Exc3** mit dem Wert **lst** werfen. Falls **m** hingegen eine **Exc3** wirft, soll **m2** den Wert -1 zurckliefern. Im verbliebenen Fall, dass **m** keine Exception wirft, soll **m2** die Rckgabe von **m** zurckliefern.

Achtung: Denken Sie ber die Reihenfolge der **catch**-Blcke nach!

Verbindliche Anforderung: Die **throws**-Klausel soll genau eine Exception-Klasse enthalten. Diese darf aber nicht Klasse **Exception** oder Klasse **Throwable** sein (sonst Punktabzug).

Referenzlsung:

```
public int m2 ( List<Integer> lst ) throws Exc1 {    // Alternativ: throws
Exc2 ODER throws Exc3
    try {
        return X.m ( lst, Optional.of(new Integer(1)) );
                                                // Boxing fr Integer ist auch ok.
    }
    catch ( Exc3 exc ) {
        return -1;
    }
    catch ( Exc1 exc ) {
        throw new Exc3 ( lst );
    }
    // Eventuell vom Compiler verlangte return's drfen fehlen.
}
```

Anmerkung:

Alternativ knnte der Rckgabewert von **m** natrlich auch in einer vor dem **try**-Block deklarierten Variable gespeichert und am Ende von **m2** zurckgeliefert werden.

Bewertungsschema:

A: Der Kopf der Methode ist erkennbar korrekt (insbesondere kein static). Nur Exc1 oder Exc2 oder Exc 3 sind als geworfene Exception-Klasse korrekt (insbesondere kein Exception und Throwable).

B: Die Konstruktion des Optional-Objektes mit of ist erkennbar korrekt.

C: B nicht nochmals werten: Der Aufruf von m mit X ist erkennbar korrekt.

D: B und C nicht nochmals werten: Der try-Block ist erkennbar korrekt gebildet.

E: Exc3 wird erkennbar vor Exc1 gefangen.

F: E nicht nochmals werten: Das Fangen und die Behandlung von Exc3 sind erkennbar korrekt.

G: Das Fangen und die Behandlung von Exc1 sind erkennbar korrekt.

H: Nur zu vergeben, wenn auch alle anderen Punkte vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

Aufgabe 3 (39 Punkte)

Diese Aufgabe besteht aus drei Teilaufgaben.

3(a):

Folgende Funktion **foo**, gegeben zwei Listen **l1** und **l2**, erstellt eine neue Liste **l**. Jedes Element an Position **p** der Liste **l** ist das Element aus Liste **l1** an Position **p** mit entweder -1, 1 oder 0 multipliziert; je nachdem, ob die Summe der Elemente von **l2** von Position **p** bis ans Ende von **l2** negativ, positiv oder Null ist. Sollte die Länge der Liste **l2** **p** unterschreiten, wird jedes Element aus **l1** ab Position **p** mit Null multipliziert. Dementsprechend hat die Ergebnisliste **l** die gleiche Länge wie **l1**.

Gegeben sei folgende Funktion in Racket:

```
( define ( foo l1 l2 )
  ( local
    (
      ( define ( bar l1 l2 acc n )
        ( cond
          [ ( empty? l1 ) empty ]
          [ ( empty? l2 ) empty ]
          [ else ( cons ( / ( + ( first l1 ) ( first l2 ) acc ) n )
                        ( bar ( rest l1 )
                              ( rest l2 )
                              ( + ( first l1 ) ( first l2 ) acc )
                              ( + n 1 ) ) ) ]
        )
      )
    )
    ( bar l1 l2 0 1 )
  )
)
```

Folgende Klasse kennen Sie aus der Vorlesung und den Übungen:

```
public class ListItem <T> {
    public T key;
    public ListItem<T> next;
}
```

Sie können davon ausgehen, dass **ListItem<T>** zwei Konstruktoren zur Verfügung stellt. Einen parameterlosen Konstruktor, der **key** sowie **next** auf **null** initialisiert, sowie einen Konstruktor mit einem Parameter **T t**, der **key** auf **t** und **next** auf **null** initialisiert. Es ist Ihnen freigestellt, welchen von beiden Konstruktoren Sie zur Lösung der Aufgabe verwenden.

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie in Java **public**-Objektmethoden **foo** und **bar** analog zum Racket-Code oben. Diese zwei Methoden gehören zur selben Klasse (diese Klasse müssen Sie nicht selbst schreiben). Die formalen Parametertypen von **foo** sind allesamt **ListItem<Integer>**. Die formalen Parametertypen von **bar** sind vom Typ **ListItem<Integer>** für **l2** und **Integer** für **acc** und **v**. Der Rückgabotyp von **foo** ist **ListItem<Integer>**, der von **bar** ist **Integer**. Im Gegensatz zum Racket-Code oben ist die Java-Methode **bar** nicht in **foo** definiert, sondern

die beiden Methoden folgen aufeinander. Sie können davon ausgehen, dass die beiden Parameter von **foo** auf korrekt gebildete lineare Listen verweisen, die zwar leer sein können, aber niemals ein Element mit Wert **null** enthalten.

Verbindliche Anforderungen:

1. Die Methoden **foo** und **bar** sind rein rekursiv, das heißt, Schleifen sind nicht erlaubt.
2. Übersetzungen in andere Datenstrukturen (Arrays, Streams etc.) sind nicht erlaubt.
3. Die Eingabelisten **l1** und **l2** von **foo** dürfen in **foo** und **bar** nicht verändert werden. Das heißt: Sie müssen die **ListItem<Integer>**-Objekte in der Ergebnisliste mit Operator **new** erzeugen (die **key**-Werte dürfen Sie hingegen mit "=" kopieren).

Referenzlösung:

```
public Integer bar(ListItem<Integer> l, Integer acc, Integer v) {
    if (l == null && acc == 0) return 0;
    else if (l == null && acc > 0) return v;
    else if (l == null && acc < 0) return -v;
    else return bar(l.next, acc + l.key, v);
}
```

Möglichkeit 1 für foo:

```
public ListItem<Integer> foo(ListItem<Integer> l1, ListItem<Integer> l2) {
    if (l1 == null) return null;

    ListItem<Integer> result = new ListItem<>();

    result.key = bar(l2, 0, l1.key);

    if (l2 == null) {
        result.next = foo(l1.next, l2);
    } else {
        result.next = foo(l1.next, l2.next);
    }

    return result;
}
```

Möglichkeit 2 für foo:

```
public ListItem<Integer> foo(ListItem<Integer> l1, ListItem<Integer> l2) {
    if(l1 == null) return null;

    ListItem<Integer> result = new ListItem<>();

    if(l2 == null) {
        result.key = 0;
        result.next = foo(l1.next, l2);
    }

    else {
        result.key = bar(l2, 0, l1.key);
        result.next = foo(l1.next, l2.next);
    }
    return result;
}
```

Anmerkungen:

Es ist keine Vorgabe, dass die Lösung Boxing für Integer $\leftrightarrow$ int verwendet, das heißt, es darf auch mit expliziten Konversionen gearbeitet werden. Sowohl Operator "==" als auch Methode clone auf Integer wäre ok.

Deckelung:

Bei 5 Punkten, falls auch nur eine Schleife vorkommt oder falls eine Liste in eine andere Art von Datenstruktur kopiert wird.

Bewertungsschema:

• *Methode bar:*

A: Der Kopf der Methode bar ist erkennbar korrekt.

B: Der Fall, dass die Liste l2 in bar gleich null ist, ist entweder erkennbar gar nicht möglich (weil in foo abgefangen) oder wird erkennbar korrekt in bar erkannt und behandelt.

C: Der zu berechnende Wert ist erkennbar korrekt bestimmt.

D: Der rekursive Aufruf von bar ist erkennbar korrekt.

E: Nur zu vergeben, wenn auch alle anderen Punkte A-D vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

• *Methode foo:*

F: Der Kopf der Methode foo ist erkennbar korrekt. Anmerkung: Falls der Kopf der Methode foo nicht hier, aber in (b) korrekt ist, wird dieser Punkt ebenfalls vergeben.

G: Der Fall, dass l_1 gleich null ist, wird erkennbar korrekt erkannt und behandelt.

H: Der Fall, dass l_2 gleich null ist, wird erkennbar korrekt erkannt und behandelt.

I: Der Aufruf von `bar` ist erkennbar korrekt und mit den korrekten aktuellen Parametern.

J: Das einzurichtende Listenelement wird erkennbar korrekt eingerichtet.

K: Das einzurichtende Listenelement wird mit korrektem Schlüsselwert versehen.

L: Das einzurichtende Listenelement wird erkennbar korrekt hinten an den bisher erstellten Teil der Ergebnisliste angehängt.

M: Der rekursive Aufruf von `foo` ist erkennbar korrekt und mit den korrekten aktuellen Parametern.

N: Die Rückgabe von `foo` ist erkennbar korrekt.

O: Nur zu vergeben, wenn auch alle anderen Punkte G-N vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

3(b):

Folgende Funktion **foo**, gegeben zwei Listen **l1** und **l2**, erstellt eine neue Liste **l**. Jedes Element an Position **p** der Liste **l** ist das Element aus Liste **l1** an Position **p** mit entweder -1, 1 oder 0 multipliziert; je nachdem, ob die Summe der Elemente von **l2** von Position **p** bis ans Ende von **l2** negativ, positiv oder Null ist. Sollte die Länge der Liste **l2** **p** unterschreiten, wird jedes Element aus **l1** ab Position **p** mit Null multipliziert. Dementsprechend hat die Ergebnisliste **l** die gleiche Länge wie **l1**.

Gegeben sei folgende Funktion in Racket:

```
( define ( foo l1 l2 )
  ( local
    (
      ( define ( bar l2 acc v)
        ( cond
          [ ( and ( empty? l2 ) ( = acc 0 ) ) 0 ]
          [ ( and ( empty? l2 ) ( > acc 0 ) ) v ]
          [ ( and ( empty? l2 ) ( < acc 0 ) ) (- v) ]
          [ else ( bar ( rest l2 ) ( + acc ( first l2 ) ) v ) ]
        )
      )
    )
  ( cond
    [ ( empty? l1 ) empty ]
    [ ( empty? l2 ) ( cons 0 ( foo ( rest l1) l2 ) ) ]
    [ else ( cons ( bar l2 0 ( first l1 ) ) ( foo ( rest l1 ) ( rest l2 ) ) ]
  ) ) ]
)
```

Folgende Klasse kennen Sie aus der Vorlesung und den Übungen:

```
public class ListItem <T> {
    public T key;
    public ListItem<T> next;
}
```

Sie können davon ausgehen, dass **ListItem<T>** zwei Konstruktoren zur Verfügung stellt. Einen parameterlosen Konstruktor, der **key** sowie **next** auf **null** initialisiert, sowie einen Konstruktor mit einem Parameter **T t**, der **key** auf **t** und **next** auf **null** initialisiert. Es ist Ihnen freigestellt, welchen von beiden Konstruktoren Sie zur Lösung der Aufgabe verwenden.

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie in Java **public**-Objektmethoden **foo** und **bar** analog zum Racket-Code oben. Diese zwei Methoden gehören zur selben Klasse (diese Klasse müssen Sie nicht selbst schreiben). Die formalen Parametertypen von **foo** sind beide **ListItem<Integer>**. Gleiches gilt für **l2** von **bar**. Die Parameter **acc** und **v** sind vom Typ **Integer**. Der Rückgabotyp von **foo** ist **ListItem<Integer>**, der von **bar** ist **Integer**. Die Java- Methode **bar** ist nicht in **foo** definiert, sondern die beiden Methoden folgen aufeinander. In **foo** verwalten Sie

zweckmäßigerweise zwei Verweise, **head** und **tail**, auf Anfang und Ende der Ergebnisliste, falls diese nicht leer ist. Sie können davon ausgehen, dass die beiden Parameter von **foo** auf korrekt gebildete lineare Listen verweisen, die zwar leer sein können, aber niemals ein Element mit Wert **null** enthalten.

Verbindliche Anforderungen:

1. Die Methoden **foo** und **bar** sind rein iterativ, das heißt, Rekursion ist nicht erlaubt.
2. Übersetzungen in andere Datenstrukturen (Arrays, Streams etc.) sind nicht erlaubt.
3. Die Eingabelisten **l1** und **l2** von **foo** dürfen in **foo** und **bar** nicht verändert werden. Das heißt: Sie müssen die **ListItem<Integer>**-Objekte in der Ergebnisliste mit Operator **new** erzeugen (die **key**-Werte dürfen Sie hingegen mit "=" kopieren).

Referenzlösung:

```
public Integer bar(ListItem<Integer> l, Integer acc, Integer v) {
    for (ListItem<Integer> p = l; p != null; p = p.next) {
        acc += p.key;
    }

    if (acc == 0) return 0;
    else if (acc > 0) return v;
    else return -v;
}

public ListItem<Integer> foo(ListItem<Integer> l1, ListItem<Integer>
l2) {
    if (l1 == null) return null;

    ListItem<Integer> head = new ListItem<>(bar(l2, 0, l1.key));
    ListItem<Integer> tail = head;

    ListItem<Integer> p2 = l2;

    for (ListItem<Integer> p1 = l1.next; p1 != null; p1 = p1.next) {
        if (p2 != null) p2 = p2.next;

        tail.next = new ListItem<>(bar(p2, 0, p1.key));
        tail = tail.next;
    }

    return head;
}
```

Deckelung:

Bei 5 Punkten, falls Rekursion vorkommt oder falls eine Liste in eine andere Art von Datenstruktur kopiert wird.

Bewertungsschema:

• Methode bar:

A: Die Elemente von l2 werden erkennbar korrekt durchlaufen.

B: Das Rückgabeergebnis von bar ist erkennbar korrekt.

C: Nur zu vergeben, wenn auch alle anderen Punkte A-B vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

• Methode foo:

D: Der Fall, dass l1 gleich null ist, wird erkennbar korrekt getestet und behandelt.

E: Der Fall, dass l2 gleich null ist, wird erkennbar korrekt getestet und behandelt.

F: Die initiale Einrichtung eines ersten Elements ist erkennbar korrekt und mit dem erkennbar korrekten Schlüsselwert.

G: Die Einrichtung von head und tail ist erkennbar korrekt.

H: Der Durchlauf durch l1 ist erkennbar korrekt (insbesondere Auslassen des ersten Elements).

I: Das einzurichtende Listenelement wird in jedem Schleifendurchlauf erkennbar korrekt eingerichtet.

J: Das einzurichtende Listenelement wird in jedem Schleifendurchlauf mit erkennbar korrektem Schlüsselwert eingerichtet.

K: Das einzurichtende Listenelement wird in jedem Schleifendurchlauf erkennbar korrekt hinten an die Ergebnisliste angehängt.

L: Der Verweis auf das Ende der Ergebnisliste (also tail) wird erkennbar korrekt aktualisiert.

M: Es wird erkennbar korrekt der Kopf der Ergebnisliste (also head) zurückgegeben.

N: Nur zu vergeben, wenn auch alle anderen Punkte D-M vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

3(c):

Folgende Funktion **foo**, gegeben zwei Listen **l1** und **l2**, erstellt eine neue Liste **l**. Jedes Element an Position **p** der Liste **l** ist das Element aus Liste **l1** an Position **p** mit entweder -1, 1 oder 0 multipliziert; je nachdem, ob die Summe der Elemente von **l2** von Position **p** bis ans Ende von **l2** negativ, positiv oder Null ist. Sollte die Länge der Liste **l2** **p** unterschreiten, wird jedes Element aus **l1** ab Position **p** mit Null multipliziert. Dementsprechend hat die Ergebnisliste **l** die gleiche Länge wie **l1**.

Gegeben sei folgende Funktion in Racket:

```
( define ( foo l1 l2 )
  ( local
    (
      ( define ( bar l2 acc v)
        ( cond
          [ ( and ( empty? l2 ) ( = acc 0 ) ) 0 ]
          [ ( and ( empty? l2 ) ( > acc 0 ) ) v ]
          [ ( and ( empty? l2 ) ( < acc 0 ) ) (- v) ]
          [ else ( bar ( rest l2 ) ( + acc ( first l2 ) ) v ) ]
        )
      )
    )
    ( cond
      [ ( empty? l1 ) empty ]
      [ ( empty? l2 ) ( cons 0 ( foo ( rest l1) l2 ) ) ]
      [ else ( cons ( bar l2 0 ( first l1 ) ) ( foo ( rest l1 ) ( rest l2 ) )
    ) ]
  ) )
)
```

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie in Java **public**-Objektmethoden **foo** und **bar** analog zum Racket-Code oben. Diese zwei Methoden gehören zur selben Klasse (diese Klasse müssen Sie nicht selbst schreiben). Die formalen Parametertypen von **foo** sind beide **List<Integer>** (gemeint ist **java.util.List**). Der formale Parametertyp von **l2** von **bar** ist vom Typ **Iterator<Integer>**. Die formalen Parametertypen von **acc** und **v** von **bar** sind vom Typ **Integer**. Der Rückgabotyp von **foo** ist **List<Integer>**. Der Rückgabotyp von **bar** ist **Integer**. Im Gegensatz zum Racket-Code oben ist die Java-Methode **bar** nicht in **foo** definiert, sondern die beiden Methoden folgen aufeinander. Sie können davon ausgehen, dass die beiden Parameter von **foo** auf korrekt gebildete lineare Listen verweisen, auch wenn diese eventuell keine Listenelemente beinhalten. Sie können ebenfalls davon ausgehen, dass diese Listen nicht **null** sind und niemals ein Element mit Wert **null** enthalten. Falls Sie in Racket eine leere Liste zurückgeben würden, geben Sie in Java eine Liste mit 0 Elementen zurück.

Verbindliche Anforderungen:

1. Auf die Elemente von **l1** und **l2** wird ausschließlich mit Iteratoren zugegriffen.
2. Übersetzungen in andere Datenstrukturen (Arrays, Streams etc.) sind nicht erlaubt.

Erinnerung: Interface **List<T>** hat eine parameterlose Objektmethode **iterator** mit Rückgabotyp **Iterator<T>**, eine Objektmethode **add** mit einem Parameter vom Typ **T**, sowie eine weitere Methode **add** mit einem Parameter vom formalen Typ **int** und einen zweiten Parameter vom formalen Typ **T**. Die Methode **add** fügt ihren Parameter hinten an die Liste an, die zweite am Index. Die boolesche Methode **isEmpty** von **List<T>** ist parameterlos.

Wichtig: Zusätzlich dürfen Sie die Funktion **listIterator(int idx)** von **List<T>** benutzen. Diese Funktion gibt einen Iterator zurück, der sich vor dem Element mit dem übergebenen Index **idx** befindet. Beispielsweise gibt der Aufruf **listIterator(0)** einen Iterator zurück, der sich vor dem ersten Element der Liste befindet, analog zu **iterator()**. Interface **Collection<T>** hat eine parameterlose Methode **size**. Interface **Iterator<T>** hat zwei parameterlose Methoden **hasNext** und **next**, wobei **hasNext** Rückgabotyp **boolean** und **next** Rückgabotyp **T** hat. Klasse **LinkedList<T>** hat einen parameterlosen Konstruktor.

Referenzlösung:

```
public Integer bar(Iterator<Integer> it, Integer acc, Integer v) {
    while (it.hasNext()) acc += it.next();

    if (acc == 0) return 0;
    else if (acc > 0) return v;
    else return -v;
}

public List<Integer> foo(List<Integer> l1, List<Integer> l2) {
    List<Integer> result = new LinkedList<>();

    if (l1.isEmpty()) return result;

    int idx = 0;
    for (Iterator<Integer> it1 = l1.iterator(); it1.hasNext();) {
        result.add(bar(l2.listIterator(idx), 0, it1.next()));
        if (idx != l2.size()) ++idx;
    }

    return result;
}
```

Deckelung:

Bei 5 Punkten, falls nicht allein über Iteratoren auf die Elemente der beiden Eingabelisten zugegriffen wird oder falls eine Liste in eine andere Art von Datenstruktur kopiert wird.

Bewertungsschema:

- *Methode bar:*

A: Der Kopf der Methode bar ist erkennbar korrekt.

B: Der Durchlauf mittels Iterator durch übergebenen Iterator ist erkennbar korrekt.

C: Das Bestimmen des Ergebniswertes ist erkennbar korrekt.

D: Nur zu vergeben, wenn auch alle anderen Punkte A-C vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

- *Methode foo:*

E: Der Kopf der Methode foo ist erkennbar korrekt.

F: Die Einrichtung der Ergebnisliste ist erkennbar korrekt.

G: Die Rückgabe einer Liste mit 0 Elementen für den Fall, dass l1 0 Elemente besitzt, ist erkennbar korrekt.

H: Der Durchlauf durch l1 mittels Iterator ist erkennbar korrekt.

I: Das Einfügen in die Ergebnisliste ist erkennbar korrekt.

J: Der an bar übergebene Iterator ist korrekt mittels listIterator und dem passenden Index gebildet.

K: Nur zu vergeben, wenn auch alle anderen Punkte F-J vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

Aufgabe 4 (27 Punkte)

Diese Aufgabe besteht aus drei Teilaufgaben.

Ansage während der Klausur: '(1 2 3) ist eine Kurzform für (list 1 2 3).

4(a):

Gegeben sei folgende Funktion in Racket:

```
(define (foobar foo bar init)
  (local
    ((define (h acc a b)
      (cond
        [(empty? a) acc]
        [(foo (first a) b) (h (bar (first a) acc) (rest a) b)]
        [else (h acc (rest a) b)]
      )
    ))
    (lambda (a b) (h init a b))
  )
)
```

Wie Sie sehen, sind **foo** und **bar** Funktionen. Gegeben sei zudem der folgende beispielhafte Aufruf von **foobar**:

```
( (foobar (lambda (x b) (> x b)) + 0) '(1 2 3) 1 )
```

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie in Java ein nichtgenerisches funktionales **public**-Interface **Foo** mit funktionaler Methode **apply**, das auf **foo** passt, wobei **Integer** der Zahlentyp ist.

Schreiben Sie zudem eine nichtgenerische **public**-Klasse **MyFoo**, die **Foo** implementiert und dasselbe Ergebnis zurückliefert wie der Lambda-Ausdruck für **foo** im obigen beispielhaften Aufruf.

Erinnerung: Methode **get** von **List** liefert das Listenelement an der im aktuellen Parameter angegebenen Position zurück, wobei Positionen wie üblich mit 0 beginnen.

Referenzlösung:

```
public interface Foo {
    boolean apply (Integer l, Integer b);
}

public class MyFoo implements Foo {
    public boolean apply (Integer l, Integer b) { return l > b; }
}
```

Bewertungsschema:

- *Interface Foo:*

A: Die funktionale Methode ist erkennbar korrekt.

B: Der Kopf der Definition des Interface ist erkennbar korrekt.

- *Klasse MyFoo:*

C: MyFoo der Vergleich $l > b$ wird korrekt durchgeführt. Dies muss aber *nicht* über einen lamda-Ausdruck geschehen. Die einzige Vorgabe ist, dass die Funktion ein äquivalentes Ergebnis zum Racket Code liefert.

D: C nicht nochmals werten: Der zurückzuliefernde Ausdruck in apply ist korrekt. Dies muss aber *nicht* über einen lamda-Ausdruck geschehen. Die einzige Vorgabe ist, dass die Funktion ein äquivalentes Ergebnis zum Racket Code liefert.

E: Der Kopf der Methode apply ist erkennbar korrekt (insbesondere public).

F: Der Kopf der Klasse ist erkennbar korrekt.

G: Nur zu vergeben, wenn auch alle anderen Punkte A-F vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

4(b):

Gegeben sei folgende Funktion in Racket:

```
(define (foobar foo bar init)
  (local
    ((define (h acc a b)
      (cond
        [(empty? a) acc]
        [(foo (first a) b) (h (bar (first a) acc) (rest a) b)]
        [else (h acc (rest a) b)]
      )
    ))
    (lambda (a b) (h init a b))
  )
)
```

Wie Sie sehen, sind **foo** und **bar** Funktionen. Gegeben sei zudem der folgende beispielhafte Aufruf von **foobar**:

```
( (foobar (lambda (x b) (> x b)) + 0) '(1 2 3) 1 )
```

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie in Java ein generisches funktionales **public**-Interface **Bar** mit generischem Typparametern **T** und **U**, das auf **bar** passt, wobei die funktionale Methode **apply** heißen soll.

Schreiben Sie in Java eine nichtgenerische **public**-Klasse **MyBar**, die **Bar** mit Instanziierung **Integer** für beide Typparameter implementiert und dasselbe Ergebnis zurückliefert wie der Lambda-Ausdruck für **bar** im obigen beispielhaften Aufruf.

Ansage während der Klausur: Im letzten Satz muss es "Ausdruck für **bar**" statt "Lambda-Ausdruck für **bar**" heißen.

Referenzlösung:

```
public interface Bar<T, U> {
    public T apply(T x, U y);
}

public class MyBar implements Bar<Integer, Integer> {
    public Integer apply(Integer x, Integer y) {
        return x + y;
    }
}
```

Bewertungsschema:

• *Interface Bar:*

A: Die funktionale Methode ist erkennbar korrekt. Der Rückgabebetyp der funktionalen Methode spielt keine Rolle in der Bewertung.

B: Der Kopf der Definition des Interface ist erkennbar korrekt.

• *Klasse MyBar:*

C: Der logische Ausdruck ist erkennbar korrekt.

D: C nicht nochmals werten: Der zurückzuliefernde Ausdruck ist korrekt.

E: Der Kopf der Methode apply ist erkennbar korrekt.

F: Der Kopf der Klasse ist erkennbar korrekt.

G: Nur zu vergeben, wenn auch alle anderen Punkte A-F vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

4(c):

Gegeben sei folgende Funktion in Racket:

```
(define (foobar foo bar init)
  (local
    ((define (h acc a b)
      (cond
        [(empty? a) acc]
        [(foo (first a) b) (h (bar (first a) acc) (rest a) b)]
        [else (h acc (rest a) b)]
      )
    ))
    (lambda (a b) (h init a b))
  )
)
```

Wie Sie sehen, sind **foo** und **bar** Funktionen. Gegeben sei zudem der folgende beispielhafte Aufruf von **foobar**:

```
( (foobar (lambda (x b) (> x b)) + 0) '(1 2 3) 1 )
```

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie in Java eine nicht-generische **public**-Klassenmethode **foobar**, die auf Basis von **Foo** und **Bar** die obige Racket-Funktion **foobar** eins-zu-eins nach Java übersetzt. Rückgabetypp von **foobar** ist **java.util.function.BiFunction** mit passenden Typparametern. Zahlentyp ist **Integer** und Listentyp ist **List<Integer>**.

Sie können in Java eine rekursive Hilfsfunktion **h** definieren, die die Parameter der Racket-Funktion **h** sowie die Parameter der Racket-Funktion **foobar** annimmt. Die Java Funktion **rest** ist gegeben mit der selben Funktionalität wie im Racket-Code:

```
static <T> List<T> rest(List<T> l)
```

Schließlich übersetzen Sie den obigen beispielhaften Racket-Ausdruck nach Java.

Verbindliche Anforderung: Der Lambda-Ausdruck im obigen Racket-Code muss in einen Lambda-Ausdruck in Java übersetzt werden.

Erinnerung: Die Java-Interfaces **Foo** und **Bar** aus Teil (a) und (b) passen auf **foo** und **bar** im obigen beispielhaften Aufruf. Die funktionalen Methoden von **Foo** und **Bar** heißen **apply**. Methode **get** von **List** liefert das Listenelement an der im Parameter angegebenen Position zurück, wobei Positionen wie üblich mit 0 beginnen.

Interface **BiFunction** hat drei generische Typparameter **A**, **B** und **C** (in dieser Reihenfolge) und repräsentiert mathematische Funktionen $A \times B \rightarrow C$. Die funktionale Methode von **BiFunction** heißt **apply**.

Referenzlösung:

```
static Integer h(Foo foo, Bar<Integer, Integer> bar, Integer acc,
List<Integer> a, Integer b) {
    if (a.size() == 0) return acc;
    else if (foo.apply(a.get(0), b)) return h(foo, bar,
bar.apply(a.get(0), acc), rest(a), b);
    else return h(foo, bar, acc, rest(a), b);
}

public static BiFunction<List<Integer>, Integer, Integer> foobar(Foo
foo, Bar<Integer, Integer> bar, Integer init) {
    return (List<Integer> a, Integer b) -> {
        return h(foo, bar, init, a, b);
    };
}

foobar(new MyFoo(), new MyBar(), 0).apply(Arrays.asList(1, 2, 3), 1);
```

Anmerkung:

Im Rahmen der Java-Regeln dürfen die Lambda-Ausdrücke selbstverständlich verkürzt/unverkürzt formuliert sein.

Bewertungsschema:

• Definition von foobar:

A: Die Parameterliste in der Definition von foobar ist erkennbar korrekt.

B: Der Rückgabetypp von foobar ist erkennbar korrekt.

C: A+B nicht nochmals werten: Der Kopf der Methode foobar ist erkennbar korrekt.

D: Der Aufruf von foo ist erkennbar korrekt.

E: F nicht nochmals werten: Der Aufruf von bar ist erkennbar korrekt.

F: D+E nicht nochmals werten: Der Lambda-Ausdruck in der Definition von foobar ist erkennbar korrekt.

• Aufruf von foobar:

G: Der Rumpf des Lambda-Ausdrucks für foo ist erkennbar korrekt.

H: G nicht nochmals werten: Der Lambda-Ausdruck für foo ist erkennbar korrekt.

I: Der Rumpf des Lambda-Ausdrucks für bar ist erkennbar korrekt.

J: I nicht nochmals werten: Der Lambda-Ausdruck für bar ist erkennbar korrekt.

K: G-J nicht nochmals werten: Der Aufruf von foobar ist erkennbar korrekt. Hier ist egal, ob foobar mit Klasse X, einer anderen Klasse oder ohne eine Klasse aufgerufen wurde.

L: Der Aufruf der Rückgabe von foobar ist erkennbar korrekt.

M: Nur zu vergeben, wenn auch alle Punkte A-L vergeben werden und auch darüber hinaus nichts falsch ist: Die Definition von foobar ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

Aufgabe 5 (24 Punkte)

Diese Aufgabe besteht aus zwei Teilaufgaben.

5(a):

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie eine **public**-Klassenmethode **foobar**, die zwei Parameter **a** und **b** vom Typ "Array von **int**" und Rückgabotyp "Array von **int**" hat (die Klasse, zu der **foobar** gehört, müssen Sie nicht schreiben). Ihre Methode **foobar** kann ohne Prüfung davon ausgehen, dass **a** und **b** auf **int**-Arrays mit Länge größer 0 verweisen und dass jede Komponente tatsächlich ein gültiger **int**-Wert ist.

Für jeden Index **i** in der Schnittmenge der Indexbereiche von **a** und **b** (also für jedes $i \in \{0, \dots, \min\{a.length, b.length\}\}$), an dem ein Element an dieser Stelle der beiden Arrays den gleichen Betrag hat, enthält das zurückgelieferte Array eine Komponente mit diesem Element zuerst als Negation des Betrags und dann den Betrag selbst als Inhalt. Demnach ist dieses Element zweimal im zurückgelieferten Array enthalten. Die Reihenfolge dieser Komponenten ist dieselbe wie in **a** und **b**. Darüber hinaus enthält das zurückgelieferte Array keine Komponenten.

Beispiel : $a = [1, \underline{5}, \underline{-7}, 2, \underline{0}, -8], b = [2, \underline{-5}, \underline{-7}, -1, \underline{0}, 9, 1] \rightarrow [-5, 5, -7, 7, -0, 0]$

Verbindliche Anforderungen:

1. Lösen Sie die oben genannte Aufgabe rein iterativ, das heißt, Rekursion ist nicht erlaubt.
2. Sie müssen auf Arrays arbeiten. Streams, Listen usw. sind nicht erlaubt.

Erinnerung: Arraytypen haben eine Klassenkonstante `length`. Klasse `Math` hat eine Klassenmethode `abs`, die einen Wert von Typ `int` entgegennimmt und den Betrag des Werts als `int` zurückgibt. Klasse `Math` hat zusätzlich eine Klassenmethode `min`, die zwei Zahlen entgegennimmt und die kleinere der beiden Zahlen zurückgibt. Die akzeptierten Typen von `min` sind `int`, `double`, `float`, und `long`.

Referenzlösung:

```
public static int[] foobar (int[] a, int[] b){
    int numberOfValues = 0;
    int maxIndex = Math.min (a.length, b.length); // Alternativ if, switch
oder Bedingungsoperator
    // Alternativ kann auf maxIndex verzichtet werden und das Minimum direkt
in die Schleife kommen.
    for (int i = 0; i < maxIndex; i++){
        if (Math.abs(a[i]) == Math.abs(b[i])) // Alternativ abs    oder a[i] ==
b[i] || a[i] == -b[i]
            numberOfValues++;
        }

    int [] result = new int[2 * numberOfValues];
    int indexInResult = 0;
    for (int i = 0; i < maxIndex; i++){
        if (Math.abs(a[i]) == Math.abs(b[i])){
            result[indexInResult] = -Math.abs(a[i]);
            result[indexInResult+1] = Math.abs(a[i]);
            indexInResult += 2;
        }
    }
    return result;
}
```

Deckelung:

bei 5 Punkten falls Rekursion vorkommt.

Anmerkung:

Es dürfen auch andere Methoden von Math verwendet werden. Es dürfen Hilfsmethoden verwendet werden, solange die Hilfsmethoden nicht die Anforderungen an die Aufgabenstellung - wie hier iterativ - unterwandern.

Bewertungsschema:

A: Der Kopf der Methode foobar ist erkennbar korrekt. Anmerkung: Falls nicht hier korrekt, aber in 5(b), soll dieser Punkt ebenfalls vergeben werden.

B: Die Prüfung auf Gleichheit der beiden int-Werten ist an mindestens einer der beiden notwendigen Stellen erkennbar korrekt.

C: Der Rumpf der Schleife für das Bestimmen der Länge des zurückzuliefernden Arrays ist erkennbar korrekt.

D: C nicht nochmals werten: Die Länge des zurückzuliefernden Arrays wird korrekt berechnet und das Array korrekt mit dieser Länge eingerichtet.

E: Die Verwaltung des jeweils aktuellen Index im zurückzuliefernden Array (indexInResult in der Referenzlösung) ist erkennbar korrekt.

F: E nicht nochmals werten: Es werden auf erkennbar korrekte Weise die korrekten Werte in das zurückzuliefernde Array geschrieben.

G: Die Reihenfolge im zurückzuliefernden Array ist erkennbar korrekt.

H: Es werden keine weiteren Zeichen außer den geforderten in das zurückzuliefernde Array geschrieben.

I: Nur zu vergeben, wenn auch alle anderen Punkte A-H vergeben werden und auch darüber hinaus nichts falsch ist: Die Lösung ist erkennbar insgesamt syntaktisch, semantisch und logisch vollständig korrekt.

5(b):

Sie brauchen keine **import**- oder **package**-Anweisungen zu schreiben.

Schreiben Sie eine **public**-Klassenmethode **foobar**, die zwei Parameter **a** und **b** vom Typ "Array von **int**" und Rückgabotyp "Array von **int**" hat (die Klasse, zu der **foobar** gehört, müssen Sie nicht schreiben). Ihre Methode **foobar** kann ohne Prüfung davon ausgehen, dass **a** und **b** auf **int**-Arrays mit Länge größer 0 verweisen und dass jede Komponente tatsächlich ein gültiger **int**-Wert ist.

Für jeden Index **i** in der Schnittmenge der Indexbereiche von **a** und **b** (also für jedes $i \in \{0, \dots, \min\{\mathbf{a.length}, \mathbf{b.length}\}\}$), an dem ein Element an dieser Stelle der beiden Arrays den gleichen Betrag hat, enthält das zurückgelieferte Array eine Komponente mit diesem Element zuerst als Negation des Betrags und dann den Betrag selbst als Inhalt. Demnach ist dieses Element zweimal im zurückgelieferten Array enthalten. Die Reihenfolge dieser Komponenten ist dieselbe wie in **a** und **b**. Darüber hinaus enthält das zurückgelieferte Array keine Komponenten.

Für die Länge des zurückzuliefernden Arrays schreiben Sie eine rekursive Hilfsmethode **foo**, die **a** und **b** sowie ein **int** namens **index** als Parameter hat und die Anzahl der Indizes größer/gleich **index** in **a** und **b** zurückliefert, von denen das Element in das Ergebnisarray gemäß obiger Spezifikation zu übernehmen ist.

Für die Befüllung des zurückzuliefernden Arrays mit Zeichen schreiben Sie eine rückgabelose rekursive Hilfsmethode **bar** mit **a** und **b**, einem **int**-Array **result** sowie zwei **int** namens **indexInAAndB** und **indexInResult** als Parameter. Dabei soll **indexInResult** immer der jeweils nächste Index in **result** sein, an den ein Zeichen zu kopieren ist.

Beispiel : $a = [1, \underline{5}, \underline{-7}, 2, \underline{0}, -8], b = [2, \underline{-5}, \underline{-7}, -1, \underline{0}, 9, 1] \rightarrow [-5, 5, -7, 7, -0, 0]$

Verbindliche Anforderungen:

1. Lösen Sie die oben genannte Aufgabe rein rekursiv, das heißt, Schleifen sind nicht erlaubt.
2. Sie müssen auf Arrays arbeiten. Streams, Listen usw. sind nicht erlaubt.

Erinnerung: Arraytypen haben eine Klassenkonstante **length**. Klasse **Math** hat eine Klassenmethode **abs**, die einen Wert von Typ **int** entgegennimmt und den Betrag des Werts als **int** zurückgibt. Klasse **Math** hat zusätzlich eine Klassenmethode **min**, die zwei Zahlen entgegennimmt und die kleinere der beiden Zahlen zurückgibt. Die akzeptierten Typen von **min** sind **int**, **double**, **float**, und **long**.

Referenzlösung:

```
private static int foo ( int[ ] a, int[ ] b, int index ) {
    if ( index >= a.length || index >= b.length ) // Alternativ mit Abfrage
        über Math.min
        return 0;
    return foo ( a, b, index + 1 ) + ( Math.abs(a[index]) ==
Math.abs(b[index]) ) ? 1 : 0 );
    // Alternativ switch oder Ausdruck mit Bedingungsoperator
    // Alternativ abs oder a[i] == b[i] || a[i] == -b[i]
}

private static void bar ( int[ ] a, int[ ] b, int[ ] result, int
indexInAAndB, int indexInResult ) {
    if ( indexInAAndB >= a.length || indexInAAndB >= b.length )
        // Alternativ mit Abfrage über Math.min
        return;
    if ( Math.abs(a[indexInAAndB]) == Math.abs(b[indexInAAndB]) ) {
        result[indexInResult] = -Math.abs(a[indexInAAndB]);
        result[indexInResult+1] = Math.abs(a[indexInAAndB]);
        indexInResult += 2;
    }
    bar ( a, b, result, indexInAAndB+1, indexInResult );
}

public static int[ ] foobar ( int[ ] a, int[ ] b ) {
    int[ ] result = new int [ 2 * foo ( a, b, 0 ) ];
    bar ( a, b, result, 0, 0 );
    return result;
}
```

Deckelung:

bei 5 Punkten falls Schleifen vorkommen.

Anmerkung:

Es dürfen auch andere Methoden von Math verwendet werden. Es dürfen Hilfsmethoden verwendet werden, solange die Hilfsmethoden nicht die Anforderungen an die Aufgabenstellung - wie hier iterativ - unterwandern.

Bewertungsschema:

- *Hilfsmethode zur Bestimmung der Länge des zurückzuliefernden Arrays: foo*

A: Der Rekursionsabbruch wird erkennbar korrekt abgefragt und behandelt.

B: Der rekursive Aufruf von der Hilfsmethode ist erkennbar korrekt.

C: Die Rückgabe im Rekursionsschritt ist erkennbar in jedem Fall korrekt.

D: Nur zu vergeben, wenn auch alle Punkte A-C vergeben werden und auch darüber hinaus nichts falsch ist: Die gesamte Methode foo ist erkennbar syntaktisch, semantisch und logisch vollständig korrekt.

- *Hilfsmethode zur Bestimmung der Elemente im zurückzuliefernden Array: bar*

E: Der Rekursionsabbruch wird erkennbar korrekt abgefragt und behandelt.

F: Es wird erkennbar genau im richtigen Fall ein Zeichen nach result kopiert, und dieser Kopierakt ist auch erkennbar korrekt.

G: Der rekursive Aufruf von der Hilfsmethode (Indizes!) ist erkennbar korrekt in dem Fall, dass das aktuelle Zeichen zu kopieren ist. Anmerkung: In der Referenzlösung wird die Unterscheidung für G und H durch Erhöhung von indexInResult im Falle von G realisiert. Stattdessen könnte die Hilfsmethode aber beispielsweise auch zweimal separat aufgerufen werden, einmal mit indexInResult und einmal mit indexInResult+2.

H: Der rekursive Aufruf von der Hilfsmethode (Indizes!) ist erkennbar korrekt in dem Fall, dass das aktuelle Zeichen nicht zu kopieren ist.

I: Nur zu vergeben, wenn auch alle Punkte E-H vergeben werden und auch darüber hinaus nichts falsch ist: Die gesamte Methode foo ist erkennbar syntaktisch, semantisch und logisch vollständig korrekt.

- *Hilfsmethoden:*

J: Die Köpfe der Hilfsmethoden sind erkennbar korrekt (insbesondere static).

- *Methode foobar:*

K: Der Kopf von foobar ist erkennbar korrekt.

L: Der Aufruf von einer geeigneten Hilfsmethode zur Bestimmung der Größe des Arrays ist in foobar erkennbar korrekt und mit geeigneten aktuellen Parameterwerten.

M: Das zurückzuliefernde Array wird erkennbar korrekt und mit der korrekten Länge eingerichtet.

N: Der Aufruf von einer geeigneten Hilfsmethode in foobar zur Berechnung der Elemente im zurückzuliefernden Array ist erkennbar korrekt und mit geeigneten Parameterwerten.

O: Nur zu vergeben, wenn auch alle Punkte K-N vergeben werden und auch darüber hinaus nichts falsch ist: Die gesamte Methode foobar ist erkennbar syntaktisch, semantisch und logisch vollständig korrekt.

Aufgabe 6 (16 Punkte)

Diese Aufgabe besteht aus vier Teilaufgaben.

6(a): Methoden

Aufgaben zu Methoden

Frage 1: In welchem Fall ist das Überladen von Methoden in Java überhaupt relevant?

Antwort 1 und Bewertungsschema:

A: Das Überladen von Methoden in Java ist relevant, wenn Sie Methoden mit ähnlicher Funktionalität aber unterschiedlichen Parametern benötigen.

Frage 2: Wie unterscheidet sich das Überladen von dem Überschreiben von Methoden?

Antwort 2 und Bewertungsschema:

B: Das Überladen von Methoden erfolgt, wenn Methoden mit dem gleichen Namen, aber unterschiedlichen Parametern in derselben Klasse definiert werden. Das Überschreiben von Methoden tritt auf, wenn eine Subklasse eine Methode ihrer Basisklasse mit derselben Signatur implementiert.

Frage 3: Wovon hängt die Sichtbarkeit einer Methode ab?

Antwort 3 und Bewertungsschema:

C: Die Sichtbarkeit von einer Methode hängt vom Modifier ab.

Frage 4: Wie können Subklassen in Java die Sichtbarkeit von Methoden ihrer Basisklassen verändern und welche Einschränkungen gelten dabei?

Antwort 4 und Bewertungsschema:

D: In Java kann die Sichtbarkeit einer Methode von einer Basisklasse in einer Subklasse erweitert werden, sofern sie nicht als private deklariert ist. (Die Sichtbarkeit kann jedoch nicht verringert werden.)

6(b): Abstrakte Klassen und Interfaces

Aufgaben zu abstrakte Klassen und Interfaces

Frage 1: Ist eine Klasse abstrakt, genau dann, wenn sie mindestens eine abstrakte Methode hat? Begründen Sie Ihre Antwort!

Antwort 1 und Bewertungsschema:

A: Nein.

B: Abstrakte Klassen müssen keine (abstrakten) Methoden enthalten.

Frage 2: Nennen Sie einen Vorteil, den Interfaces gegenüber abstrakte Klassen haben.

Antwort 2 und Bewertungsschema:

C: Sie erlauben Mehrfachvererbung.

6(c): Arrays, Listen, Streams

Aufgaben zu Arrays, Listen, Streams

Aufgabe 1: Nennen Sie ein Alleinstellungsmerkmal, dass die folgenden Konzepte jeweils gegenüber den anderen beiden Konzepten haben:

Arrays
Listen
Streams

Beispielhaft bedeutet das: Nennen Sie eine Eigenschaft, die **Arrays** haben, aber **Listen** und **Streams** nicht haben.

Antwort 1 und Bewertungsschema:

Diese Referenzantwort enthält alle möglichen Antworten, eine Auswahl (wie in der Aufgabenstellung gefordert) wurde nicht vorgenommen.

A: Mögliche Alleinstellungsmerkmale für "Arrays":

- Feste Länge
- Primitive Datentypen möglich
- Keine generischen Typen möglich
- Eigener Syntax

B: Mögliche Alleinstellungsmerkmale für "Listen":

- Hinzufügen von Elementen
- Entfernen von Elementen
- Nicht-unendliche Dynamische Größe

C: Mögliche Alleinstellungsmerkmale für "Streams":

- Unendliche Größe möglich
- Effizientes "method chaining"
- Nur einmal "verwendbar"
- Größe wird als long abgespeichert
- Hilfsstruktur/Keine Datenstruktur

Aufgabe 2: Entscheiden Sie für jede der folgenden Zeilen, ob der Java-Compiler eine Fehlermeldung werfen würde, oder nicht. (Gehen Sie davon aus, dass der Code Teil eines Methodenkörpers ist):

```
Number[] n = new Integer[42];  
List<Number> lstNum = new LinkedList<Double>();
```

Antwort 2 und Bewertungsschema:

A: Es wurde korrekt erkannt, dass Zeile 1 zu keiner Fehlermeldung führt, also "durch den Compiler geht".

B: Es wurde korrekt erkannt, dass Zeile 2 zu einer Fehlermeldung führt.

6(d): *IntPredicate*

Aufgaben zu *IntPredicate*

Frage 1: Wie unterscheidet sich das Interface **IntPredicate** von Interfaces im Generellen?

Antwort 1 und Bewertungsschema:

A: Es ist ein Functional Interface.

B: Nicht jedes Interface ist ein Functional Interface.

Frage 2: Wozu kann **IntPredicate** genutzt werden?

Antwort 2 und Bewertungsschema:

C: Als Functional Interface ist es nutzbar, um lambda-Ausdrücke in Java nachzubilden.

Frage 3: Nennen Sie mögliche Ein- und Ausgabetypen von **IntPredicate**.

Antwort 3 und Bewertungsschema:

D: Eingabetyp ist int und Ausgabetyyp ist boolean.

